



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Master Test Plan Document

Versione 1.0

Team

Choaib Goumri
Mattia Gallucci

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
2	Strategie di Testing	2
2.1	Testing di Unità	2
2.2	Testing di Integrazione	2
3	Organizzazione delle funzionalità testate	3
3.1	Componenti Incluse dal Testing	3
3.2	Componenti escluse dal Testing	3
3.3	Coverage	3
4	Criteri di successo	4

1 Introduzione

Il seguente documento descrive il **Master Test Plan** del progetto **CodeSmile** definisce la strategia complessiva, l'approccio metodologico e le attività pianificate per il processo di testing del sistema software.

L'**obiettivo** principale del documento è quello di stabilire un quadro strutturato e coerente delle attività di verifica e validazione, raggiungendo un adeguato livello di qualità e affidabilità.

Il **piano di test** copre i diversi livelli di testing previsti per il progetto, includendo:

- **Testing di unità**, finalizzato alla verifica delle singole componenti software;
- **Testing di integrazione**, volto a validare le interazioni tra i principali moduli del sistema;
- **Testing di sistema**, condotto dal punto di vista dell'utente e basato sul comportamento osservabile del sistema;

Il documento fornisce inoltre indicazioni sugli obiettivi di copertura, sui criteri di accettazione e sulle condizioni di superamento delle attività di test, costituendo un riferimento fondamentale per la pianificazione, l'esecuzione e la valutazione del processo di testing lungo l'intero ciclo di vita del progetto.

2 Strategie di Testing

2.1 Testing di Unità

L'attività di **unit testing** ha l'obiettivo di verificare il corretto funzionamento delle singole unità software (classi, funzioni e moduli) in maniera isolata, individuando precocemente eventuali difetti di implementazione.

I test di unità sono progettati seguendo un approccio **white-box**, con particolare attenzione alla struttura interna del codice e ai flussi di controllo. In particolare, i casi di test sono definiti in modo da esercitare i principali rami logici del programma, includendo condizioni normali, casi limite e scenari di errore.

La suite di unit test è organizzata in modo coerente con la struttura del codice sorgente: i test sono suddivisi in package o directory che rispecchiano i moduli dell'applicazione, garantendo una chiara tracciabilità tra componenti testate e casi di test.

L'esecuzione dei test di unità è supportata da strumenti automatici di testing e di **misurazione della copertura del codice**. L'obiettivo è assicurare una copertura significativa delle parti core del sistema, riducendo il rischio di difetti nelle fasi successive di integrazione e di sistema.

2.2 Testing di Integrazione

L'attività di **testing di integrazione** ha l'obiettivo di verificare il corretto funzionamento delle interazioni tra le diverse unità software già validate tramite i test di unità. In questa fase, l'attenzione si concentra sui punti di interfaccia tra i moduli e sui flussi di dati che attraversano più componenti del sistema.

Il testing di integrazione è finalizzato a individuare difetti quali errori di comunicazione tra moduli, uso non corretto delle interfacce, gestione errata delle dipendenze e incoerenze nei dati scambiati tra i componenti.

I test sono progettati seguendo un approccio incrementale, integrando progressivamente i moduli principali del sistema e verificando il corretto comportamento delle funzionalità risultanti.

3 Organizzazione delle funzionalità testate

3.1 Componenti Incluse dal Testing

Tutte le componenti della nuova architettura saranno oggetto di testing di unità e di integrazione al fine di garantire elevati livelli di qualità, affidabilità e manutenibilità del sistema.

3.2 Componenti escluse dal Testing

Al momento non rientrano nel piano le funzionalità di gestione e addestramento del modello di Intelligenza Artificiale, con particolare riferimento agli aspetti di Machine Learning.)

3.3 Coverage

L'attività di testing è accompagnata da un processo di **misurazione della copertura**, con l'obiettivo di **valutare** in modo quantitativo **l'efficacia dei test eseguiti** e il grado di esercitazione del codice e delle funzionalità del sistema.

Per i **test di unità**, l'obiettivo principale è garantire una copertura significativa delle componenti core del sistema. In particolare, viene posta attenzione alla **branch coverage**, al fine di assicurare che i principali rami logici del codice vengano eseguiti almeno una volta durante i test. L'obiettivo minimo prefissato è il raggiungimento di una copertura pari ad almeno l'**80%** per i moduli principali dell'applicazione.

Per i **test di integrazione**, la copertura è valutata in termini della percentuale di codice esercitata dai flussi che coinvolgono più componenti cooperanti. Pur non essendo l'obiettivo principale di questa fase, si mira a ottenere una copertura indicativa superiore al **50%**, sufficiente a garantire una verifica adeguata delle interazioni tra i moduli.

4 Criteri di successo

Il processo di testing è considerato completato con successo quando risultano soddisfatti i seguenti criteri di accettazione:

- Coverage non inferiore all'80% sulle parti fondamentali dell'architettura software, verificata nell'ambito delle attività di test unitari.
- Completamento con **esito positivo** dell'intero insieme di casi di test di unità, di integrazione e di sistema.
- **Mancanza** di **anomalie** o problemi rilevanti emersi nel testing.

Il soddisfacimento di tali criteri consente di considerare il **sistema idoneo** al rilascio e conforme agli obiettivi di qualità definiti per il progetto.